StagePass

Student Workshop 8w

Version 7.1 Y

Project Description

Welcome to the StagePass workshop! You'll be working with a Spring Boot REST API for managing concert ticket bookings.

The application will allow users to browse upcoming concerts, purchase tickets, manage their bookings, and look up venue information.

There are two parts of this workshop:

- Bug tickets (**required**)

- New feature requests (optional)

Getting Started

- **Running the Application**
  1. Open MySQL workbench and run the following query:

     ```
     CREATE SCHEMA `stagepass`;
     ```

  2. Open the `debug-project` folder in your IDE
  3. Change the `application.properties` to reflect the correct username and password for the database
  4. Run the application
  5. The API will start on `http://localhost:8080`

- **Seed data**

  The database is pre-populated with:

  - **3 Artists:** The Rolling Stones, Dua Lipa, Tiësto
  - **3 Venues:** AFAS Live, Ziggo Dome, Paradiso (all in Amsterdam)
  - **6 Concerts:** 2 past concerts and 4 future concerts
  - **3 Bookings:** spread across different concerts

Tips

- **Start with the bugs**

  They’re easier and will help you understand the codebase.

- **Use the MySQL Workbench** to review the database

- **Test after each fix**

  Don’t try to fix everything at once.

- **Read the error messages**

  Spring Boot gives helpful error details.

- **Follow the layers : Controller → Service → Repository**

  Bugs are in specific layers; knowing which layer helps narrow your search.

- **Use your debugger**

  Set breakpoints in the service methods and step through the code.

Part 1 – Bug Tickets (Required)

The application has **5 bugs** hiding in different layers. Your job is to find and fix as many as you can. Use the API endpoints to observe the buggy behavior, then trace through the code to find the root cause.

- **API Endpoints Reference**

Method	URL	Description
GET	`/api/artists`	All artists
GET	`/api/artists/{id}`	Single artist
POST	`/api/artists`	Create artist
PUT	`/api/artists/{id}`	Update artist
DELETE	`/api/artists/{id}`	Delete artist
GET	`/api/venues`	All venues
GET	`/api/venues/{id}`	Single venue
POST	`/api/venues`	Create venue
PUT	`/api/venues/{id}`	Update venue
DELETE	`/api/venues/{id}`	Delete venue
GET	`/api/concerts`	All concerts
GET	`/api/concerts/{id}`	Single concert
POST	`/api/concerts`	Create concert
PUT	`/api/concerts/{id}`	Update concert
DELETE	`/api/concerts/{id}`	Delete concert
GET	`/api/concerts/artist/{artistId}`	Concerts by artist
GET	`/api/concerts/upcoming`	Future concerts
GET	`/api/concerts/{id}/summary`	Concert booking
GET	`/api/bookings`	All bookings
GET	`/api/bookings/{id}`	Single booking
GET	`/api/bookings/concert/{concertId}`	Bookings for a concert
POST	`/api/bookings`	Create booking
DELETE	`/api/bookings/{id}`	Cancel booking

BUG-01 – Wrong HTTP Status on Create

Field	Details
Ticket ID	BUG-01
Type	Bug
Layer	Controller
Endpoint	POST endpoints

- **Symptom:** When you create a new booking (or artist, venue, concert), the response status code is wrong

- **How to reproduce:**
  1. Send a POST request to `/api/bookings` with a valid booking body
  2. Check the HTTP status code in the response
  3. What status code do you get? What should it be?

- **Acceptance criteria:**
  - `POST /api/bookings` returns **201 Created** when a booking is successfully saved
  - `POST /api/artists`, `POST /api/venues`, and `POST /api/concerts` also return **201 Created**

- **Hint:** Look at the controller layer. What does the REST convention say about successful resource creation?

BUG-02 – Available Seats Not Decremented

Field	Details
Ticket ID	BUG-02
Type	Bug
Layer	Service
Endpoint	POST `/api/bookings`

- **Symptom:** After creating a booking, the concert's available seats remain unchanged.

- **How to reproduce:**
  1. Check a concert's available seats: `GET /api/concerts/3`
  2. Create a booking for that concert: `POST /api/bookings`
  3. Check the concert again: `GET /api/concerts/3`
  4. Are the available seats updated?

- **Example POST body for creating a booking:**

```
{
  "customerName": "Test Student",
  "customerEmail": "test@example.com",
  "concert": { "id": 3 },
  "numberOfTickets": 2
}
```

- **Acceptance criteria:**
  - After POST `/api/bookings` with `numberOfTickets=2`, the concert's `availableSeats` decreases by 2
  - `GET /api/concerts/{id}` reflects the updated seat count

- **Hint:** Look at the service layer. What should happen to the concert when a booking is saved?

BUG-03 – Delete Returns Wrong Status

Field	Details
Ticket ID	BUG-03
Type	Bug
Layer	Controller
Endpoint	DELETE endpoints

- **Symptom:** Deleting a booking (or artist, venue, concert) returns the wrong HTTP status code.

- **How to reproduce:**
  1. Send `DELETE /api/bookings/1`
  2. Check the HTTP status code
  3. What status code do you get? What should it be for a successful deletion with no response body?
- **Acceptance criteria:**
  - `DELETE /api/bookings/{id}` returns **204 No Content** with an empty response body
  - The same fix is applied consistently to all delete endpoints (artists, venues, concerts)

- **Hint:** What HTTP status code is appropriate when a resource is successfully deleted and there is no response body?

BUG-04 – Total Price not Computed

Field	Details
Ticket ID	BUG-04
Type	Bug
Layer	Service
Endpoint	POST `/api/bookings`

- **Symptom:** The `totalPrice` field in a newly created booking is always `0.00`, regardless of how many tickets are booked.

- **How to reproduce:**
  1. Create a booking for a concert with `ticketPrice` of 65.00 and `numberOfTickets` of 3
  2. Look at the `totalPrice` in the response
  3. Is it correct? What should it be?
- **Acceptance criteria:**
  - POST `/api/bookings` with `numberOfTickets=3` and a concert `ticketPrice` of 65.00 returns `totalPrice` of **195.00**
  - The fix uses `BigDecimal` arithmetic (not `double` or `int`) to avoid rounding issues

- **Hint:** Look at the service layer. How is `totalPrice` being calculated (or not)?

BUG-05 – Concert Filter Returns All Bookings

Field	Details
Ticket ID	BUG-05
Type	Bug
Layer	Service / Repository
Endpoint	POST `/api/bookings/concert/{concertId}`

- **Symptom:** The endpoint to get bookings for a specific concert returns **all bookings** in the system instead of filtering by concert.

- **How to reproduce:**
  1. Call `GET /api/bookings/concert/3` – note the results
  2. Call `GET /api/bookings/concert/4` – note the results
  3. Are they different? They should be!

- **Acceptance criteria:**
  - `GET /api/bookings/concert/3` returns only bookings for concert ID 3
  - `GET /api/bookings/concert/4` returns only bookings for concert ID 4
  - Results for different concerts should not be identical

- **Hint:** The repository has the right method. Is the service using it?

Part 2 – Feature Requests (Bonus Challenge)

After fixing the bugs, challenge yourself and implement the following 5 features.

Each one requires changes in multiple layers (repository, service, and/or controller).

Please keep in mind, this is a bonus part. The features are getting increasingly difficult as you go progress.

FEAT-01 – Overbooking Protection

Field	Details
Ticket ID	FEAT-01
Type	Feature
Layer	Service, Controller
Endpoint	POST `/api/bookings`

- **Description:** Prevent bookings when not enough seats are available. A customer should not be able to book 100 seats for a concert that only has 5 seats left.

- **Requirements**
  - In `BookingService.createBooking()`, check whether `concert.getAvailableSeats() >= booking.getNumberOfTickets()`
  - If not enough seats, throw a custom `InsufficientSeatsException`
  - Create a `@RestControllerAdvice` exception handler that returns **409 Conflict** with a JSON body containing a `message` field

- **How to test:**
  1. Find a concert with limited seats (concerts 4 and 5 have a few remaining)
  2. Try to book more tickets than available
  3. You should get a 409 response with an error message

- **Acceptance criteria:**
  - POST `/api/bookings` with more tickets than available seats returns **HTTP 409 Conflict**
  - Response body contains a `message` field explaining the rejection
  - Booking with exactly the available number of seats succeeds

FEAT-02 – Search Concerts by Artist

Field	Details
Ticket ID	FEAT-02
Type	Feature
Layer	Repository, Service, Controller
Endpoint	POST `/api/concerts/artist/{artistId}`

- **Description:** Users want to see all concerts featuring a specific artist

- **Requirements:**
  - Add `List<Concert> findByArtistId(Long artistId)` to `ConcertRepository`
  - Add `getConcertsByArtist(Long artistId)` to `ConcertService`
  - Add `GET /api/concerts/artist/{artistId}` to `ConcertController`

- **How to test:**
  1. `GET /api/concerts/artist/1` — should return Rolling Stones concerts only
  2. `GET /api/concerts/artist/999` — should return an empty array, not a 404

- **Acceptance criteria:**
  - GET `/api/concerts/artist/1` returns only concerts for artist ID 1
  - An artist with no concerts returns an empty JSON array `[]`

FEAT-03 – Upcoming Concerts Filter

Field	Details
Ticket ID	FEAT-03
Type	Feature
Layer	Repository, Service, Controller
Endpoint	POST `/api/concerts/upcoming`

- **Description:** Users want to see only future concerts, not ones that already happened.

- **Requirements:**
  - Add `List<Concert> findByDateAfterOrderByDateAsc(LocalDate date)` to `ConcertRepository`
  - Add `getUpcomingConcerts()` to `ConcertService` that calls `findByDateAfterOrderByDateAsc(LocalDate.now())`
  - Add `GET /api/concerts/upcoming` to `ConcertController`

- **How to test:**
  1. `GET /api/concerts/upcoming` – should not include past concerts
  2. Results should be sorted by date ascending (earliest first)

- **Acceptance criteria:**
  - GET `/api/concerts/upcoming` does not return concerts with a date in the past
  - Results are sorted by date ascending (earliest first)

FEAT-04 – Concert Booking Summary

Field	Details
Ticket ID	FEAT-04
Type	Feature
Layer	Service, Controller
Endpoint	GET `/api/concerts/{id}/summary`

- **Description:** Concert organizers want a summary showing seats sold, seats remaining, and total revenue.
- **Requirements:**
  - Create a `ConcertSummary` record (or class) with fields: `concertId`, `concertTitle`, `totalSeats`, `seatsBooked`, `availableSeats`, `totalRevenue`
  - Calculate `seatsBooked` by summing `numberOfTickets` across all bookings for that concert
  - Calculate `totalRevenue` by summing `totalPrice` across those bookings
  - Return **404** if the concert doesn't exist
- **How to test:**
  1. `GET /api/concerts/3/summary` — should show booking statistics
  2. `GET /api/concerts/9999/summary` — should return 404
- **Acceptance criteria:**
  - Response includes `concertId`, `concertTitle`, `totalSeats`, `seatsBooked`, `availableSeats`, `totalRevenue`
  - `seatsBooked + availableSeats` equals `totalSeats`
  - GET `/api/concerts/9999/summary` returns **HTTP 404**

FEAT-05 – Input Validation

Field	Details
Ticket ID	FEAT-05
Type	Feature
Layer	Model, Controller
Endpoint	POST `/api/bookings`, POST `/api/concerts`

- **Description:** The API should reject invalid input using Bean Validation annotations.

- **Requirements:**

  Add these validation annotations to the **Booking** model:

  - `customerName`: `@NotBlank`
  - `customerEmail`: `@NotBlank` and `@Email`
  - `numberOfTickets`: `@Min(1)` and `@Max(10)`

  Add these validation annotations to the **Concert** model:

  - `title`: `@NotBlank`
  - `date`: `@NotBlank` and `@FutureOrPresent`
  - `totalSeats`: `@Positive`
  - `ticketPrice`: `@NotNull` and `@DecimalMin("0.01")`

  Add `@Valid` to the `@RequestBody` parameters in the relevant controller methods.

- **How to test:**
  1. POST `/api/bookings` with `customerName: ""` — should get 400
  2. POST `/api/bookings` with `customerEmail: "not-an-email"` — should get 400
  3. POST `/api/bookings` with `numberOfTickets: 0` or `numberOfTickets: 11` — should get 400
  4. POST `/api/concerts` with a past date — should get 400
  5. Valid input should still work and return 201 (after fixing BUG-01!)

- **Acceptance criteria:**
  - Invalid input returns **HTTP 400 Bad Request**
  - Valid input creates the resource successfully